PROJECT BLOODHOUND

A Rule-Based Framework for Multi-Source Log Analysis and Incident Reconstruction

Technical Report

Portfolio Project | Python Log Detective Lab

Abstract

[Write this section last. In no more than one page, summarize the problem, Bloodhound's deterministic approach, evidence sources, principal capabilities, blind-validation method, and high-level results.]

Keywords

Python, log analysis, incident reconstruction, Suricata, authentication analysis, cross-source correlation, digital evidence

Table of Contents

Abstract 2
1 Project Overview 5
1.1 Motivation 5
1.2 Project Objectives 5
1.3 Scope and Boundaries 6
2 Dataset and Experimental Design 7
2.1 Dataset Description 7
2.2 Blind-Analysis Method 7
3 Bloodhound Architecture 9
3.1 Core Components 9
3.2 Data Flow and Module Interfaces 10
3.3 Design Characteristics and Constraints 12
3.4 Prerequisite and State Management ..... **Error! Bookmark not defined.**
4 Log Parsing and Normalization 14
4.1 Log Parsing 14
4.2 Normalization 15
5 Detection Methodology 16
5.1 Authentication Detection 16
5.2 Network Detection 16
5.3 Cross-Source Correlation Detection 16
6 Unified Timeline and Reporting 18
6.1 Unified Timeline 18
6.2 Reporting 18
7 Results and Discussion 19
7.1 Results 19
7.2 Discussion 19
8 Conclusion 21
Appendix A – Dataset Generation 22
Appendix B – Answer Key 31
Appendix C – Bloodhound Analysis 32

Appendix D - Internal Record Schemas .....**Error! Bookmark not defined.**
Appendix E - Complete Bloodhound Analysis Output .....**Error! Bookmark not defined.**
Appendix F - Ground-Truth Comparison .....**Error! Bookmark not defined.**
Appendix G - File Hashes and Version Metadata .....**Error! Bookmark not defined.**
Appendix H - Repository Structure .....**Error! Bookmark not defined.**

.

1 Project Overview

1.1 Motivation

The primary motivation for Project Bloodhound was to add a practical, entry-level cybersecurity project to the portfolio. Project Bloodhound is part of a trio of existing portfolio projects including, a password-recovery project and a distributed network security monitoring and mathematical threat modeling project. While the latter two projects attempted to demonstrated technical depth, experimental design, and cybersecurity research skills, their specialized and research-oriented nature did not directly demonstrate many of the day-to-day analytical skills expected in an entry-level cybersecurity role.

Project Bloodhound was developed to fill that gap. The project's focus is on tasks representative of a junior security analysis: importing heterogeneous log files, parsing and normalizing event records, identifying suspicious authentication and network activity, correlating evidence across multiple sources, constructing a unified timeline, and producing a structured analysis report. Implementing these capabilities in Python also provided practical experience with data structures, file processing, configuration management, timestamp handling, rule-based detection, and defensive error handling.

Bloodhound therefore complements the earlier portfolio projects. While the earlier projects emphasize research and specialized technical investigation, Bloodhound demonstrates the ability to build and apply a reproducible workflow for examining security logs and organizing incident evidence.

1.2 Project Objectives

The primary objective of Project Bloodhound was to demonstrate practical competency with foundational cybersecurity analysis tasks. Bloodhound was not intended to explore a novel research question or develop a mathematically sophisticated technique, unlike previous projects. It was designed to show that the skills developed through introductory cybersecurity training could be applied in a complete, functioning project.

The project's specific objectives were to:

- Import and process host and network security logs from multiple sources,
- Parse heterogeneous log formats and normalize relevant event data,
- Identify suspicious authentication and network activity using deterministic detection rules,
- Correlate related evidence across authentication logs, system logs, and Suricata network records,
- Organize security-relevant events into a unified chronological timeline,

- Produce a structured and reproducible analysis report without overstating what the available evidence established, and
- Develop a modular Python application with configuration management, persistent analysis state, input validation, and defensive error handling.

Taken together, these objectives were intended to demonstrate readiness for entry-level cybersecurity work involving log review, alert investigation, evidence correlation, incident documentation, and basic security automation.

1.3 Scope and Boundaries

Project Bloodhound was developed as a controlled laboratory application for analyzing a simulated cybersecurity incident. Its scope was limited to four Linux hosts, one network sensor, and three evidence types: authentication logs, system logs, and Suricata `eve.json` logs. The application processes previously collected log files rather than monitoring systems or network traffic in real time.

Bloodhound performs the following functions:

- Imports and parses the supported log formats,
- Normalizes relevant fields into consistent Python data structures,
- Applies predefined rules to identify authentication and network anomalies,
- Correlates qualifying events across multiple evidence sources,
- Organizes relevant events into a unified chronological timeline, and
- Produces a structured report containing the completed analytical results.

Bloodhound is deterministic and relies on rule-based parsing and analysis. Within that scope, the program may classify activity as a potential candidate for brute-force authentication, credential spraying, lateral movement, or data exfiltration. However, these classifications are intended only as warnings and require further investigation to establish their validity.

Bloodhound excludes generalized incident interpretation. For example, the program does not opine on attacker identity or intent, establish causation from temporal proximity or establish contents of an outbound transfer. The program also does not construct a narritive of an incident. Interpretation of the analysis is properly the responsibility of a human analyst in this entry level project.

The program was designed and evaluated using a modest simulated dataset, and the implemented detection thresholds and correlation rules were selected accordingly. The sealed ground-truth record (see below) was excluded from the analytical process so that Bloodhound's findings could be produced independently and compared with the known incident only during final validation.

2 Dataset and Experimental Design

2.1 Dataset Description

Bloodhound was developed and evaluated using a simulated collection of host and network logs. To provide an independent test, Google Gemini was used to generate a dataset containing malicious and benign events for a small Linux environment, together with an answer key identifying the threat signatures embedded in the dataset. (see Appendix A and B). The dataset covered the four-hour period from July 26, 2026, at 18:00 UTC through July 26, 2026, at 22:00 UTC.

The simulated environment corresponding to the dataset consisted of four Linux systems, identified as node1 through node4, and one Suricata network sensor. Each Linux node contributed an authentication log and a general system log. The network sensor contributed a Suricata eve.json log.

The three supported logs types serve different analytical purposes:

Evidence type	Source	Purpose
auth.log	Linux nodes	Authentication attempts, successful logins, usernames, source addresses, and SSH activity
syslog	Linux nodes	System and process activity occurring near qualifying authentication events
eve.json	Suricata sensor	Network connections, flow measurements, SSH metadata, alerts, and source and destination endpoints

The dataset included both malicious and benign background events. That is, Bloodhound was required to distinguish events meeting its detection and correlation criteria from unrelated records, rather than treating every logged event as evidence of an incident.

2.2 Blind-Analysis Method

The dataset was analyzed using a blind-analysis method. As noted above, an answer key was generated together with the dataset. The answer key identified the threat signatures included in the auth.log, syslog, and eve.json files and provided a narrative describing the simulated attack activity. Notably, the answer key remained sealed throughout program development, rule implementation, debugging, and the initial analysis.

Bloodhound used only the evidence available in the node and sensor logs. Detection rules were based on observable characteristics such as authentication failures, the number of targeted users or hosts, network destinations, transferred-byte counts, Suricata alerts, and temporal relationships

between events. The rules were not constructed by consulting the known incident and then searching for its predetermined features.

The analysis procedure consisted of the following stages:

1. Import and parse the data files.
2. Review the imported evidence and confirm the analytical time range.
3. Analyze the Data for Threat Signatures
4. Apply the authentication-detection rules.
    4.1. Apply the network-detection rules.
    4.2. Correlate qualifying events across the available evidence sources.
    4.3. Construct a unified timeline from the selected evidence.
5. Generate and preserve the final structured analysis report.
6. Compare the preserved results with the sealed ground truth during final validation.

3 Bloodhound Architecture

3.1 Core Components

Bloodhound is organized as a modular Python application consisting of a main program, six supporting modules, and a TOML configuration file. The main program controls the user interface, imports the configured log files, maintains the application state, and orchestrates the supporting modules in the required sequence. Each supporting module performs a distinct stage of the analysis.

The principal components are summarized below:

Component	Primary Responsibility
bloodhound.py	Main. Reads the configuration, imports the log files, performs initial parsing, manages the menu, maintains application state, and orchestrates the analysis sequence
auth_parser.py	Applies template patterns to authentication records and extracts fields such as service, outcome, event type, username, source IP address, port, and command information
auth_analysis_detections.py	Applies authentication-detection rules and identifies successful logins occurring after activity from a flagged source
net_analysis_detections.py	Applies network-detection rules to Suricata records
correlation_analysis.py	Associates qualifying authentication events (auth.log) with subsequent system (syslog) and network activity (eve.json)
unified_timeline.py	Converts selected findings into a common chronological representation and removes duplicate events
analysis_report.py	Produces a structured text report from the completed analysis stored in the application state
config.toml	Defines the dataset location, node and sensor names, supported log filenames, and node-to-IP mapping

The main program provides four user operations: Import Log Files; Display an Evidence Overview; Analyze Log Data; and Generate the Analysis Report. The Import Log Files operation reads the configured `auth.log`, `syslog`, and `eve.json` files. Authentication and system logs are initially separated into timestamp, host, process, process identifier, and message fields. Suricata records are read as newline-delimited JSON and retain their event-specific fields. The imported records are combined in a pandas DataFrame stored as `log_data`.

The Evidence Overview operation summarizes the imported records by host and log type. For each source, it reports the number of records and the earliest and latest timestamps. It also calculates the complete time range represented by the imported evidence. This operation provides a basic check that the expected files were imported and that their timestamps correspond to the intended analysis period.

The Analyze Log Data operation controls the substantive analysis. It invokes authentication analysis, network analysis, cross-source correlation, and unified-timeline generation in sequence. Combining these stages under one menu option simplified the user interface while preserving the separation of the underlying analytical functions.

The Analysis Report operation consolidates the completed results into a structured, human-readable report. It does not repeat the detection process or introduce new analytical conclusions. Instead, it retrieves the results already stored by the preceding stages and presents their relevant fields, measurements, classifications, evidence sources, and timestamps.

3.2 Data Flow and Module Interfaces

Bloodhound uses a sequential data-processing pipeline. Information produced by one stage is stored in a shared application-state dictionary, “data_state”, where it becomes available to the stages that follow. This provides a common interface through which the modules exchange Pandas DataFrames, detection dictionaries, correlation chains, and report text.

At program startup, bloodhound.py reads config.toml. The configuration identifies the dataset directory and the expected combination of hosts, sensors, and log types. Bloodhound uses these values to construct a list of expected evidence-file paths. The configuration also supplies the node-to-IP mapping needed to associate hostname-based host records with IP-based network records.

During import, Bloodhound reads each configured file line by line. Host logs undergo an initial general parse, while each Suricata line is decoded as a JSON object. The resulting records are combined, and their timestamps are converted into pandas datetime values.

The authentication-analysis stage selects the auth.log records from the imported evidence and passes them to auth_parser.py. The parser compares each record against defined authentication-event patterns. Recognized records are enriched with structured fields describing the authentication service, outcome, event type, username, source address, and other available details.

The structured authentication records are then passed to the detection functions in auth_analysis_detections.py. Each detection function returns a DataFrame containing only the events or grouped activity meeting that rule’s criteria. The resulting DataFrames are stored in the auth_detections dictionary according to detection type. Bloodhound also compares flagged authentication sources with successful SSH logins and stores qualifying post-failure successes separately as compromise_success.

The network-analysis stage selects the eve.json records from the imported evidence. The network functions examine these records for port scanning, multi-host targeting, large outbound transfers, and native Suricata alerts. As with the authentication analysis, each rule returns a separate DataFrame, and the results are stored by detection type in network_detections.

Cross-source correlation begins with a qualifying successful authentication event from a source previously flagged by an authentication rule. This event provides the initial host, username, source address, and timestamp for a candidate correlation chain. Bloodhound resolves the authenticated host's internal IP address through the explicit node-to-IP mapping and defines a 60-minute forward correlation window.

Within that window, the correlation module searches for three categories of related evidence: selected system activity recorded on the authenticated host; large outbound transfers or Suricata alerts involving the host's internal IP address; and subsequent SSH activity originating from the authenticated host toward another monitored node.

Matching records are retained as structured event dictionaries with their individual timestamps, descriptions, event types, and evidence sources. Each completed chain receives an identifier such as COR-001. A chain is retained as a cross-source result only when it contains evidence from more than one log source. This prevents authentication activity alone from being presented as cross-source correlation.

The unified-timeline module then draws from the previously completed authentication, network, and correlation results. It converts selected findings into a common schema containing a timestamp, attributed host, event type, description, evidence source, and correlation identifier. Events belonging to a correlation chain retain that chain's identifier, while standalone detections are marked as uncorrelated. The module removes duplicate representations and sorts the remaining records chronologically. It performs no additional detection.

Finally, the report module reads the stored evidence summary, authentication findings, network findings, correlation chains, and unified timeline. These results are converted into a deterministic text report. Because the report is generated from stored results, the same application state produces the same report content.

The principal data_state entries are:

Key	Stored information
log_data	All imported host and sensor records
log_data_meta	Record counts and time ranges by host and log type
auth_data	Structured authentication records
auth_detections	Authentication findings grouped by rule
compromise_success	Successful logins associated with previously flagged sources
network_data	Imported Suricata records selected for network analysis
network_detections	Network findings grouped by rule
node_ip_map	Configured node-to-IP mapping
correlated_chains	Structured cross-source evidence chains

unified_timeline	Deduplicated chronological findings
analysis_report	Final structured report text

3.3 Design Characteristics and Constraints

Bloodhound was designed as a deterministic, offline log-analysis application appropriate for the scale and scope of the project. Its architecture emphasizes modularity, reproducibility, transparent rules, and separation between automated detection and human interpretation. Modularity was achieved by separating authentication parsing, authentication detection, network detection, correlation, timeline construction, and report generation into individual Python modules.

The application uses deterministic rules rather than machine learning or generalized behavioral analysis. Each finding results from defined field requirements, grouping criteria, thresholds, time windows, or event patterns. The same input files, configuration, and program version should therefore produce the same results.

Dataset-specific information is partially separated from the program logic through `config.toml`. File locations, participating nodes, expected log types, sensor names, and node-to-IP mappings can be changed without modifying the parsing modules. Detection thresholds and correlation criteria remain defined within their respective Python functions.

The node-to-IP mapping is an important design feature. Authentication logs identify the system producing the record by hostname while representing the connecting system as a source IP address. Suricata records generally identify both endpoints by IP address. Bloodhound resolves these different forms of identity explicitly instead of inferring that an authentication source address belongs to the host that recorded the event.

The program also separates detection, correlation, timeline construction, and reporting. A detection identifies activity satisfying an individual rule. Correlation determines whether selected findings are eligible to be associated across evidence sources. Timeline construction reorganizes already selected evidence chronologically. Report generation presents the completed results. Neither the timeline nor the report introduces new detections or converts temporal association into a conclusion about causation.

The program has several notable constraints. It processes previously collected files in batches and does not monitor live systems or network traffic. All imported records and analytical results are held in memory for the current program session; no database or more elaborate management system is implemented. Application state is not preserved automatically after the program exits. Furthermore, the program assumes the configured files adhere to the supported Linux and Suricata formats and does not provide a general plug-in interface for additional evidence types.

Finally, the program's classifications remain analytical warnings. A rule match can identify activity consistent with brute-force authentication, credential spraying, lateral movement, or a large outbound transfer, but the program does not determine identity, intent, compromise, or the

contents of transferred data.

4 Log Parsing and Normalization

4.1 Log Parsing

Bloodhound must convert each log entry from raw text into a form that Python can analyze. Because the Linux host logs and Suricata network log use different formats, the program applies a parsing method appropriate to each source.

The Linux `auth.log` and `syslog` files contain one event per line. Bloodhound first divides each line into four general parts: timestamp, hostname, process information, and message. The process information is then separated into a process name and process identifier (`pid`). This produces the following common fields:

Field	Description
timestamp	Date and time of the event
host	System that produced the log entry
process	Process or service that recorded the event
pid	Process identifier, when present
message	Original event message
log_type	auth.log or syslog

The `message` field is preserved because it contains details that cannot be extracted reliably by the general parser. For authentication records, Bloodhound performs a second parsing step using the templates defined in `auth_parser.py`. The templates use regular expressions to recognize supported event formats and extract the security-relevant values contained in each message.

The authentication templates cover several common Linux services and event types, including:

- OpenSSH authentication successes, failures, and disconnections,
- `systemd-logind` session events,
- Pluggable Authentication Module (`pam_unix`) events,
- `sudo` and `su` privilege or identity changes,
- PolicyKit authorization activity,
- User and group account changes, and
- Scheduled-task sessions created by `cron`.

When a message matches a template, the parser records the service, outcome, event type, and any additional fields available in that message. For example, a failed SSH authentication message may provide a username, source IP address, and source port. A `sudo` message may instead provide the user, target user, working directory, terminal, and command. Bloodhound applies the templates in sequence and uses the first applicable template to classify the event.

For Suricata `eve.json` files, the program reads each line with Python's JSON parser and adds the configured sensor name and the `eve.json` log type. Fields already present in the Suricata record, such as source and destination IP addresses, ports, event type, flow information, and alert information, are retained for later analysis.

4.2 Normalization

The imported logs do not all describe events in the same way. Linux logs contain mostly unstructured message text, while Suricata records already contain named JSON fields. Authentication events also vary by service. Correspondingly, the program uses partial normalization. The purpose is not to force every record into an identical format. Instead, the program creates consistent fields for information that has the same analytical meaning while preserving the fields specific to each evidence source.

All imported records receive a `host` and `log_type` value. The timestamp field is converted to a pandas datetime value using mixed-format timestamp handling. This permits records from different sources to be filtered by time, compared with one another, placed inside correlation windows, and sorted into a unified timeline.

Authentication records receive a standard set of additional columns: auth_service (service associated with the event}; outcome (result such as success, failed, or session_opened); event_type (general category of authentication or account activity); username (user associated with the event); target_user (account affected or requested by the user); src_ip (remote source address, when available); port (remote source port, when available); session_id (login-session identifier); command (command associated with an administrative event); tty (terminal associated with the event); and calling_uid (ser identifier that initiated a session or action)

These columns are created before the authentication templates are applied. When a template matches, the parser fills the fields available for that event. Fields that do not apply remain empty. Unrecognized authentication messages are retained in the DataFrame with their original message rather than being discarded, although they are not classified into one of the supported authentication event types.

When the host and sensor records are combined, pandas creates a shared set of columns containing all fields found in the imported evidence. A record uses the fields that apply to its source and contains empty values in fields that do not apply. For example, an authentication record may contain "username" and "outcome", while a Suricata flow record may contain destination-address and transferred-byte fields.

This normalization provides the consistency needed for the later detection rules without removing the differences between the evidence sources.

5 Detection Methodology

5.1 Authentication Detection

Bloodhound applies rule-based detections to the normalized `auth.log` records. The rules examine failed SSH attempts by source IP address, username, destination host, and time.

1. Single-Source Brute Force: Four or more failed SSH attempts against the same username and host within four minutes
2. Credential Spraying: One source attempts no more than two passwords against each of at least five usernames within 24 hours
3. Multi_Host Targeting: One source produces failed SSH attempts against at least three hosts within 15 minutes

Bloodhound also checks whether a source flagged by an authentication rule later completes a successful SSH login. The successful login must occur after the detected failure activity. This does not prove that an account was compromised, but it raises the priority of the finding for further investigation.

5.2 Network Detection

Bloodhound applies four detection rules to the Suricata eve.json records.

1. Port Scan: One source contacts at least ten ports on one destination within five minutes
2. Multi-Host Targeting Minutes: One source contacts the same port on at least three destination hosts within 15 minutes
3. Large Outbound Transfer: A source sends at least 8,000,000 bytes to a destination
4. Suricata Alert: Suricata records the event as an alert

These rules identify network activity that may require investigation. For example, a large outbound transfer may be consistent with data exfiltration, but the transferred volume alone does not establish what data was sent or whether the transfer was malicious.

5.3 Cross-Source Correlation Detection

Cross-source correlation begins with a successful SSH login from an IP address previously flagged by an authentication rule. Bloodhound then examines activity associated with the authenticated host during the following 60 minutes.

The program searches for relevant `syslog` activity, Suricata alerts, large outbound transfers, and subsequent SSH connections from the host to another monitored node. The configured node-to-IP mapping allows hostname-based host logs to be matched with IP-based network records.

Bloodhound retains a correlation chain only when it contains evidence from more than one log source. Each retained chain is assigned an identifier such as `COR-001`. The chain shows that related events occurred within the defined time window, but final interpretation remains the responsibility of the analyst.

6 Unified Timeline and Reporting

6.1 Unified Timeline

Bloodhound combines selected authentication detections, network detections, and correlated events into a unified timeline. Each timeline entry contains a timestamp, host, event type, description, evidence source, and correlation identifier when applicable.

The program removes duplicate events and sorts the remaining entries by timestamp. Events belonging to the same correlation chain retain a shared identifier, such as `COR-001`, which makes related activity easier to follow across different log sources.

The unified timeline does not perform additional detection or determine that one event caused another. Its purpose is to organize the relevant evidence into chronological order so that an analyst can review the sequence of activity.

6.2 Reporting

After the analysis is completed, Bloodhound produces a structured text report containing the evidence overview, authentication detections, network detections, correlation chains, and unified timeline.

The report is created from results already stored in the application state. Report generation does not repeat the analysis or add new conclusions. It presents the findings in a consistent format so that the results can be reviewed, preserved, and compared with the sealed answer key during final validation.

The report includes the rule classifications and supporting event details, but it does not claim that every flagged event is malicious. Final interpretation of the findings remains the responsibilit y of the analyst.

7 Results and Discussion

7.1 Results

Bloodhound's Analysis Report can be seen in Appendix C. Bloodhound detected one single-source brute-force candidate. The source IP address `198.51.100.45` made five failed SSH attempts against the `svc_web` account on `node1` between 18:14:03 and 18:14:15 UTC. The same source successfully logged in as `svc_web` at 18:14:27 UTC. No credential-spraying or authentication-based multi-host targeting was detected.

The network analysis produced two findings. Bloodhound detected an outbound transfer of 8,504,300 bytes from `node2` (`10.50.0.12`) to the external address `203.0.113.99` at 18:31:32 UTC. It also reported a Suricata alert involving a connection from `10.50.0.250` to `node3` (`10.50.0.13`) at 18:45:00 UTC. No port scan or network-based multi-host targeting met Bloodhound's detection thresholds.

Bloodhound created one cross-source correlation chain, identified as `COR-001`. The chain began with the successful login to `node1` and included subsequent `syslog` activity on that host. Relevant events included the reading of `/etc/webapp/db_backend_link.conf` and the execution of an SSH command targeting the `data_sync` account on `node2`. The chain also included routine system activity that occurred during the same correlation window.

The unified timeline placed the authentication detection, successful login, system activity, large outbound transfer, and Suricata alert in chronological order. The outbound transfer and Suricata alert remained separate findings and were not assigned to `COR-001`.

7.2 Discussion

Comparison of the Analysis Reposrt with the answer key showed that Bloodhound identified several important parts of the simulated incident, but it did not reconstruct the complete activity chain.

Incident Stage	Bloodhound Results	Assessment
Initial access	Detected repeated SSH failures followed by a successful login from 198.51.100.45	Detected
Discovery	Included the command reading /etc/webapp/db_backend_link.conf in COR-001	Detected
Lateral Movement	Included the SSH command from node1 to the data_sync account on node2, but did not include the	Partially Detected

	successful authentication record from node2	
Archive Preparation	Did not include the command that archived /var/lib/financial_vault/ on node2	Not Detected
Outbound Transfer	Detected the 8.5 MB transfer from node2 to 203.0.113.99, but did not connect it to the archive or COR-001	Partially Detected
Internal Vulnerability Scan	Reported the Suricata alert involving 10.50.0.250 and node3, but left it uncorrelated	Correctly Reported as Separate Alert.

The strongest result was the identification of the initial SSH attack and the successful login that followed it. Bloodhound also located the discovery command and the SSH command directed toward `node2`. These results showed that the authentication and system logs contained enough evidence to identify the beginning of the incident.

The main weakness was that Bloodhound did not connect the activity on `node1` with the later host and network activity on `node2`. The archive command and upload command were not included in the report, while the outbound transfer appeared only as a separate network detection. As a result, Bloodhound found evidence associated with the beginning and end of the incident but did not produce a complete correlation chain between them.

The results also showed a limitation of using a broad time window for system-log correlation. Several routine System Activity Data Collector events were added to `COR-001` because they occurred after the successful login, even though the answer key did not identify them as part of the incident. This demonstrates that events occurring close together in time are not necessarily related.

The Suricata alert involving `10.50.0.250` was identified in the answer key as unrelated vulnerability-scanning activity. Bloodhound correctly preserved the alert as a separate finding rather than adding it to the correlation chain. However, the program could not determine by itself that the activity was benign. That conclusion still required comparison with the surrounding evidence and review by the analyst.

Overall, Bloodhound successfully reduced the dataset to several important security events without claiming that it had proven a complete compromise. The comparison with the answer key showed that the individual detection rules worked reasonably well, while the cross-source correlation logic remained incomplete. Future improvement should focus on following lateral movement between monitored hosts and continuing the correlation on the destination host.

8 Conclusion

Project Bloodhound demonstrated how Python can be used to parse and normalize different log formats, apply rule-based detections, correlate events across multiple evidence sources, and organize the results into a unified timeline and report. During blind analysis, Bloodhound identified the initial SSH brute-force activity, the successful login that followed, related system activity, a large outbound transfer, and a separate Suricata alert.

Comparison with the answer key showed that Bloodhound detected several important parts of the simulated incident but did not connect the complete attack sequence across both affected hosts. In particular, the correlation logic did not fully follow the lateral movement from `node1` to `node2` or connect the later archive and outbound transfer activity to the original login. Overall, the project provided practical experience with Python, Linux and Suricata logs, detection rules, and cross-source analysis while also identifying clear areas for future improvement.

Appendix A – Dataset Generation

The Project Bloodhound data set was generated using AI. Google Gemini was provided the following prompt to generate the dataset:

You are the independent scenario generator for **Project Bloodhound — Python Log Detective Lab**.

Your job is to generate a self-contained mystery dataset consisting of realistic Linux and Suricata logs. Another AI will help me write the analysis program, but it must not know the ground truth. Do not explain the incident in your response.

DELIVERABLE

Create a downloadable ZIP archive named:

bloodhound_dataset_01.zip

The ZIP must contain:

```
bloodhound_dataset_01/
├── README.md
├── manifest.sha256
├── logs/
│   ├── collision/
│   │   ├── auth.log
│   │   └── syslog
│   ├── preimage/
│   │   ├── auth.log
│   │   └── syslog
│   ├── birthday/
│   │   ├── auth.log
│   │   └── syslog
│   ├── cipher/
│   │   ├── auth.log
│   │   └── syslog
│   └── sensor/
│       └── eve.json
└── sealed/
    └── ground_truth.md
```

Generate real plain-text files inside the ZIP. Do not merely display example snippets or place all the logs in one document.

PURPOSE OF THE DATASET

The dataset will be used by a beginner learning Python to build a multi-source log-investigation program.

The program will eventually learn to:

open and read files;

parse Linux text logs;

parse Suricata JSON Lines;

normalize events;

combine and sort events chronologically;

detect suspicious patterns;

correlate endpoint and network activity;

reconstruct a probable incident sequence;

distinguish evidence from inference;

compare its conclusions with a sealed answer key.

This is the first dataset, so make it realistic but manageable.

DATASET SIZE

Generate approximately:

35–75 lines in each auth.log;

50–110 lines in each syslog;

175–300 JSON objects in eve.json.

Aim for roughly 600–900 total events.

Every file must contain enough benign background activity that the incident is not immediately obvious, but do not bury it beneath thousands of irrelevant lines.

TIME AND FORMATTING REQUIREMENTS

Use one continuous investigation window lasting between two and four hours.

Use the year 2026 and a single timezone throughout the dataset. UTC is preferred.

For Linux logs, use a consistent RFC 3339-style timestamp, for example:

2026-07-26T19:42:17+00:00 collision sshd[1842]: Failed password for cryptlab from 10.20.0.51 port 49218 ssh2

Linux log lines should otherwise resemble authentic Ubuntu auth.log and syslog entries, including plausible:

hostnames;

process names;

process IDs;

usernames;

SSH ports;

service messages;

session creation and closure;

sudo records;

cron activity;

systemd messages;

routine network or maintenance events.

The four endpoint hostnames must be exactly:

collision

preimage

birthday

cipher

The Suricata file must be valid JSON Lines/NDJSON:

exactly one complete JSON object per line;

no surrounding JSON array;

no comments;

no Markdown;

no trailing commas.

Use realistic Suricata EVE fields where appropriate, including:

timestamp

flow_id

in_iface

event_type

src_ip

src_port

dest_ip

dest_port

proto

app_proto

flow

alert

ssh

dns

http

tls

Not every EVE event needs every field. Field variation should resemble real Suricata output, but every line must remain valid JSON.

HIDDEN INCIDENT DESIGN

Create one coherent hidden incident spanning multiple log sources.

The broad storyline must contain evidence supporting this progression:

suspicious authentication activity against collision;

eventual successful access to collision;

discovery or reconnaissance activity on collision;

discovery of information that gives the intruder a reason to investigate preimage or project_falcon;

lateral movement from collision toward preimage;

access to or preparation of material associated with project_falcon;

suspicious outbound transfer activity;

relevant Suricata observations that corroborate portions of the endpoint evidence.

The discovery step is essential. The intruder must not appear to know in advance that preimage is important. Evidence encountered on collision must explain the later interest in preimage.

These existing directories may appear naturally in the evidence:

collision: /home/cryptlab/projects/project_harbor

preimage: /home/cryptlab/projects/project_falcon

Do not relocate these directories to /opt or another location.

Choose the exact:

source addresses;

destination addresses;

usernames;

ports;

timestamps;

commands visible through logging;

filenames;

session identifiers;

flow identifiers;

transfer method;

benign explanations and distractions.

Keep those choices internally consistent across every file.

Do not place an attacker activity on preimage before the evidence on collision that motivates it.

EVIDENCE VISIBILITY

The incident must be discoverable from the supplied logs without requiring shell history, packet captures, Wazuh alerts, or files that are not included.

Because arbitrary unprivileged shell commands do not normally appear in auth.log, only expose command evidence through plausible mechanisms such as:

authentic-looking sudo records;

service logs;

audit messages forwarded to syslog;

file-monitoring records;

network evidence;

other believable log-producing activity.

Do not pretend that normal sshd logging records every command typed by a user.

Include enough evidence to support a reasonable reconstruction, but leave some uncertainty. The investigator should need to correlate several sources rather than locate one line that explains the entire incident.

BENIGN BACKGROUND ACTIVITY

Include realistic normal activity on all four endpoints, such as:

successful SSH administration;

occasional mistyped passwords;

cron jobs;

package or service activity;

systemd sessions;

routine DNS, NTP, SSH, HTTP, or TLS traffic;

ordinary internal communication;

expected outbound connections.

Include two or three plausible benign distractions, for example:

an administrator mistyping a password several times;

a routine large outbound transfer;

a vulnerability-scan-like connection from an authorized internal system;

a harmless service restart;

an unusual but legitimate login time.

At least one distraction should superficially resemble part of the real incident.

Do not make every unusual event malicious.

The unaffected or mostly unaffected hosts must still contain meaningful background activity. Do not leave birthday and cipher nearly empty merely because they are not central to the incident.

AVOID ANSWER LEAKAGE

Do not use labels such as these in the raw logs or filenames:

attacker

victim

malicious

attack_step

lateral_movement

exfiltration

ground_truth_event

suspicious_event

Do not insert artificial event IDs whose only purpose is to identify malicious records.

Do not include explanatory comments inside the logs.

Suricata alerts may contain realistic signature text, but avoid implausibly explicit signatures such as:

Confirmed attacker exfiltrating project_falcon

The logs should look like operational evidence, not a narrated story.

README.md may state only:

the project name;

the directory structure;

the log formats;

the investigation time range;

the timezone;

that eve.json uses JSON Lines;

that ground_truth.md should remain unopened until analysis is complete.

It must not identify suspicious systems, accounts, addresses, events, or the incident narrative.

Your chat response must not summarize or hint at the hidden incident. Simply provide the ZIP and a brief statement that generation and validation are complete.

GROUND-TRUTH ANSWER KEY

Place the complete solution only in:

sealed/ground_truth.md

The answer key must contain:

a concise incident summary;

the complete chronological attack sequence;

affected and unaffected systems;

identities and roles of relevant IP addresses;

relevant usernames;

the initial-access method;

the discovery evidence found on collision;

why that evidence led toward preimage;

the lateral-movement method;

the material accessed or staged;

the outbound-transfer method and destination;

the exact supporting events.

For every malicious or materially relevant event, identify:

source filename;

exact timestamp;

hostname;

source and destination addresses where applicable;

a short unique excerpt or identifying field;

what the event establishes;

whether it is direct evidence or requires inference.

Also document every deliberate benign distraction and explain why it is benign.

End the answer key with an expected high-level reconstruction and a list of reasonable uncertainties. Do not require the investigator to reach conclusions unsupported by the generated evidence.

INTERNAL CONSISTENCY CHECKS

Before creating the ZIP, silently validate all of the following:

Every required file exists.

Every eve.json line parses independently as valid JSON.

Every event timestamp falls within the declared investigation window.

Events are in chronological order within each individual file.

Hostnames are correct and consistent.

IP addresses retain the same roles throughout the dataset.

SSH source ports and session details are plausible.

A successful SSH session has corresponding network evidence where appropriate.

Lateral movement occurs only after discovery evidence on collision.

The outbound transfer occurs only after the relevant access or staging activity.

Shared Suricata flow_id values are internally consistent.

Benign distractions are included in the answer key.

Nothing outside sealed/ground_truth.md reveals the solution.

The raw logs do not refer to Wazuh or contain Wazuh-generated alerts.

manifest.sha256 contains SHA-256 hashes for all files except the manifest itself.

The ZIP opens correctly and preserves the required directory structure.

If any check fails, correct the dataset before delivering it.

Do not ask me to choose the attack details. Make the choices yourself, generate the complete dataset, validate it, and provide the downloadable ZIP without revealing the answer.

Appendix B – Answer Key

Below is the answer key produced during AI dataset generation.

Ground Truth Answer Key

Incident Summary

An external adversary (`198.51.100.45`) successfully brute-forced the `svc_web` account on `node1`. Reconnaissance on `node1` led to the discovery of `/etc/webapp/db_backend_link.conf`, which revealed `node2`'s role. The adversary pivoted to `node2` via SSH using the `data_sync` account, archived sensitive files from `/var/lib/financial_vault/`, and exfiltrated the archive to an external server (`203.0.113.99`).

Incident Sequence

1. **Initial Access**: 198.51.100.45 brute-forced `node1` via SSH (Direct Evidence: `auth.log` failed/accepted passwords).

2. **Discovery**: Intruder ran `cat /etc/webapp/db_backend_link.conf` on `node1` (Direct Evidence: `syslog` audit EXECVE). This provided the motivation to target `node2`.

3. **Lateral Movement**: SSH session established from `node1` to `node2` as `data_sync` (Direct Evidence: `auth.log` on `node2`).

4. **Preparation**: Intruder executed `tar -czf /tmp/sync_cache.tar.gz /var/lib/financial_vault/` on `node2` (Direct Evidence: `syslog` audit).

5. **Exfiltration**: Intruder used `curl` to upload the archive to `https://203.0.113.99/upload`. Corroborated by `eve.json` showing an 8.5MB outbound flow to `203.0.113.99` over port 443.

Benign Distractions

- **Vulnerability Scan**: `10.50.0.250` triggered a Suricata alert against `node3`. This is a distraction; the IP is an internal scanner (inferred from subnet) and no successful access or lateral movement followed.

Uncertainties

- The logs do not prove the exact contents of the exfiltrated vault, only that the directory was archived and a data transfer of corresponding size occurred.

Appendix C – Bloodhound Analysis

Below is the analysis report produced by Bloodhound after analysis of the various log files.

```
======================================================================
                    ANALYSIS REPORT
======================================================================

EVIDENCE ANALYZED
----------------------------------------------------------------------
  Host: node1    | Log Type: auth.log   | Records: 51    | Window: 2026-07-26 18:09:00+00:00 to 2026-07-26 19:37:00+00:00
  Host: node1    | Log Type: syslog     | Records: 53    | Window: 2026-07-26 18:04:00+00:00 to 2026-07-26 19:44:01+00:00
  Host: node2    | Log Type: auth.log   | Records: 46    | Window: 2026-07-26 18:02:00+00:00 to 2026-07-26 19:21:00+00:00
  Host: node2    | Log Type: syslog     | Records: 53    | Window: 2026-07-26 18:02:00+00:00 to 2026-07-26 19:46:01+00:00
  Host: node3    | Log Type: auth.log   | Records: 45    | Window: 2026-07-26 18:08:00+00:00 to 2026-07-26 19:36:00+00:00
  Host: node3    | Log Type: syslog     | Records: 51    | Window: 2026-07-26 18:05:00+00:00 to 2026-07-26 19:51:01+00:00
  Host: node4    | Log Type: auth.log   | Records: 45    | Window: 2026-07-26 18:07:00+00:00 to 2026-07-26 19:29:00+00:00
  Host: node4    | Log Type: syslog     | Records: 50    | Window: 2026-07-26 18:07:00+00:00 to 2026-07-26 20:01:01+00:00
  Host: sensor   | Log Type: eve.json   | Records: 7     | Window: 2026-07-26 18:14:03+00:00 to 2026-07-26 18:45:00+00:00

AUTHENTICATION DETECTIONS
----------------------------------------------------------------------
  [SINGLE-SOURCE BRUTE FORCE (T1110.001)] - 1 record(s) found:
    - Host: node1, Src IP: 198.51.100.45, User: svc_web | Failed Attempts: 5 | Range: 2026-07-26 18:14:03+00:00 to 2026-07-26 18:14:15+00:00
```

NETWORK DETECTIONS

[LARGE_TRANSFER] - 1 record(s) found:

- Src IP: 10.50.0.12, Dest IP: 203.0.113.99 | Bytes Sent: 8,504,300 | Range: 2026-07-26 18:31:32+00:00 to 2026-07-26 18:31:32+00:00

[SURICATA_ALERTS] - 1 record(s) found:

- Timestamp: 2026-07-26 18:45:00+00:00 | Src IP: 10.50.0.250, Dest IP: 10.50.0.13 | Signature: ET SCAN Suspicious inbound connection

CORRELATED ACTIVITY CHAINS

COR-001 — CORRELATED AUTHENTICATION ACTIVITY — POST-AUTHENTICATION SYSTEM ACTIVITY

Time range: 2026-07-26 18:14:27+00:00 to 2026-07-26 18:40:01+00:00 UTC

Authentication event: Successful password_auth from flagged source 198.51.100.45 as 'svc_web'

Target host: node1 (10.50.0.11)

System context: Command / service activity: USER_LOGIN pid=8429 uid=1001 auid=1001 ses=45 msg='op=login acct="svc_web" exe="/usr/sbin/sshd" hostname=198.51.100.45 addr=198.51.100.45 terminal=ssh res=success'

System context: Command / service activity: Starting System Activity Data Collector...

System context: Command / service activity: EXECVE pid=8500 uid=1001 auid=1001 ses=45 msg='argc=3 a0="cat" a1="/etc/webapp/db_backend_link.conf"'

System context: Command / service activity: EXECVE pid=8501 uid=1001 auid=1001 ses=45 msg='argc=3 a0="ssh" a1="data_sync@10.50.0.12"'

System context: Command / service activity: Starting System Activity Data Collector...

System context: Command / service activity: Starting System Activity Data Collector...

System context: Command / service activity: Starting System Activity Data Collector...

System context: Command / service activity: Starting System Activity Data Collector...

System context: Command / service activity: Starting System Activity Data Collector...

System context: Command / service activity: Starting System Activity Data Collector...

Evidence sources: auth.log, syslog

Classifications: CORRELATED AUTHENTICATION ACTIVITY, POST-AUTHENTICATION SYSTEM ACTIVITY

UNIFIED TIMELINE

TIMESTAMP (UTC) HOST EVENT TYPE EVIDENCE CORR ID DESCRIPTION

2026-07-26 18:14:15 node1 AUTH DETECTION auth.log N/A Repeated SSH failures (5 attempts from 198.51.100.45 for 'svc_web') on node1 (10.50.0.11)

2026-07-26 18:14:27 node1 SUCCESSFUL LOGIN auth.log COR-001 Login from flagged source 198.51.100.45 as 'svc_web' on node1 (10.50.0.11)

2026-07-26 18:14:27 node1 SYSTEM CONTEXT syslog COR-001 Command / service activity: USER_LOGIN pid=8429 uid=1001 auid=1001 ses=45 msg='op=login acct="svc_web" exe="/usr/sbin/sshd" hostname=198.51.100.45 addr=198.51.100.45 terminal=ssh res=success' on node1 (10.50.0.11)

2026-07-26 18:17:01 node1 SYSTEM CONTEXT syslog COR-001 Command / service activity: Starting System Activity Data Collector... on node1 (10.50.0.11)

2026-07-26 18:18:27 node1 SYSTEM CONTEXT syslog COR-001 Command / service activity: EXECVE pid=8500 uid=1001 auid=1001 ses=45 msg='argc=3 a0="cat" a1="/etc/webapp/db_backend_link.conf"' on node1 (10.50.0.11)

2026-07-26 18:18:29 node1 SYSTEM CONTEXT syslog COR-001 Command / service activity: EXECVE pid=8501 uid=1001 auid=1001 ses=45 msg='argc=3 a0="ssh" a1="data_sync@10.50.0.12"' on node1 (10.50.0.11)

2026-07-26 18:21:01 node1 SYSTEM CONTEXT syslog COR-001 Command / service activity: Starting System Activity Data Collector... on node1 (10.50.0.11)

2026-07-26 18:29:01 node1 SYSTEM CONTEXT syslog COR-001 Command / service activity: Starting System Activity Data Collector... on node1 (10.50.0.11)

2026-07-26 18:30:01 node1 SYSTEM CONTEXT syslog COR-001 Command / service activity: Starting System Activity Data Collector... on node1 (10.50.0.11)

2026-07-26 18:31:32 node2 LARGE TRANSFER eve.json N/A 8,504,300 bytes sent externally from node2 (10.50.0.12) to 203.0.113.99

2026-07-26 18:32:01 node1 SYSTEM CONTEXT syslog COR-001 Command / service activity: Starting System Activity Data Collector... on node1 (10.50.0.11)

2026-07-26 18:36:01 node1 SYSTEM CONTEXT syslog COR-001 Command / service activity: Starting System Activity Data Collector... on node1 (10.50.0.11)

2026-07-26 18:40:01 node1 SYSTEM CONTEXT syslog COR-001 Command / service activity: Starting System Activity Data Collector... on node1 (10.50.0.11)

2026-07-26 18:45:00 node3 SURICATA ALERT eve.json N/A Sensor-generated alert: ET SCAN Suspicious inbound connection on node3 (10.50.0.13)

ANALYSIS LIMITATIONS

- Analysis is strictly bounded by the provided log collection window and file integrity.
- Detection rules operate on heuristic pattern matching and signature thresholds.
- Cross-source correlations reflect temporal proximity and shared network endpoints, not definitive proof of attacker intent or confirmed system compromise.

==========

END OF REPORT

==========